EXPERIMENT-14: React TODO application with GitHub deployment

Create a TODO application in React with necessary components and deploy it into GitHub.

Objective : Build a React TODO app with add, complete, and delete functionality and deploy it to GitHub.

Tools : React, GitHub Pages, frontend components

Outcome : A deployed TODO application with task management and GitHub hosting.

Answer:

Create TodoList, TodoItem, and AddTodo components, manage them with React hooks, and deploy the static build output to GitHub Pages.

File Name : index.html

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>React TODO App</title>

<script crossorigin src="https://unpkg.com/react@18.2.0/umd/react.production.min.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.production.min.js"></script>
<script crossorigin src="https://unpkg.com/@babel/standalone@7.23.5/babel.min.js"></script>

<style>
 *{box-sizing:border-box}
 body{margin:0;font-family:Arial,sans-serif;background:#eaf2f8;color:#222}
 .app{max-width:520px;margin:40px auto;padding:0 15px}
 .card{background:#fff;padding:22px;border-radius:10px;box-shadow:0 2px 8px rgba(0,0,0,.12)}
 h1{margin:0 0 16px;text-align:center;color:#2c3e50}
 .add{display:flex;gap:8px}
 .add input{flex:1;padding:10px;border:1px solid #ccc;border-radius:5px;font-size:15px}
 button{cursor:pointer;border:0;border-radius:5px;font-size:14px}
 .add button{background:#1abc9c;color:#fff;padding:10px 18px}
 .add button:hover{background:#16a085}
 ul{list-style:none;padding:0;margin:16px 0 0}
```

```
 li{display:flex;align-items:center;gap:10px;padding:10px;border-bottom:1px solid #eee}
 li span{flex:1;word-break:break-word}
 li.done span{text-decoration:line-through;color:#999}
 li input{width:18px;height:18px;cursor:pointer}
 .del{background:#e74c3c;color:#fff;padding:5px 10px}
 .footer{display:flex;justify-content:space-between;align-items:center;margin-top:14px;font-
     size:14px;flex-wrap:wrap;gap:8px}
 .filters button{background:#ecf0f1;padding:5px 10px;margin-right:4px}
 .filters button.active{background:#2c3e50;color:#fff}
 .clear{background:none;color:#e74c3c;text-decoration:underline}
 .empty{text-align:center;color:#888;padding:18px 0}
 .error{color:#c0392b;font-size:13px;margin-top:6px}
</style>
</head>
<body>
<div id="root"></div>

<script type="text/babel">
const { useState, useEffect } = React;

// ---------- AddTodo component ----------
function AddTodo({ onAdd }) {
  const [text, setText] = useState('');
  const [error, setError] = useState('');

  const submit = e => {
    e.preventDefault();
    if (!text.trim()) return setError('Please type a task');
    onAdd(text.trim());
    setText('');
    setError('');
  };

  return (
    <form onSubmit={submit}>
      <div className="add">
```

```
      <input value={text} onChange={e => setText(e.target.value)} placeholder="What needs to be
      done?" />
      <button type="submit">Add</button>
    </div>
    {error && <div className="error">{error}</div>}
   </form>
  );
}

// ---------- TodoItem component ----------
function TodoItem({ task, onToggle, onDelete }) {
  return (
   <li className={task.done ? 'done' : ''}>
    <input type="checkbox" checked={task.done} onChange={() => onToggle(task.id)} />
    <span>{task.text}</span>
    <button className="del" onClick={() => onDelete(task.id)}>Delete</button>
   </li>
  );
}

// ---------- TodoList component ----------
function TodoList({ tasks, onToggle, onDelete }) {
  if (tasks.length === 0) return <div className="empty">No tasks here 🎉</div>;
  return (
   <ul>
    {tasks.map(t => (
     <TodoItem key={t.id} task={t} onToggle={onToggle} onDelete={onDelete} />
    ))}
   </ul>
  );
}

// ---------- Filters component ----------
function Filters({ filter, setFilter }) {
  return (
   <div className="filters">
```

```
    {['All', 'Active', 'Completed'].map(f => (
      <button key={f} className={filter === f ? 'active' : ''} onClick={() => setFilter(f)}>{f}</button>
    ))}
   </div>
  );
}

// ---------- App (state lives here) ----------
function TodoApp() {
  const [tasks, setTasks] = useState(() => {
   try { return JSON.parse(localStorage.getItem('todos')) || []; } catch { return []; }
  });
  const [filter, setFilter] = useState('All');

  // Save to localStorage whenever tasks change
  useEffect(() => { localStorage.setItem('todos', JSON.stringify(tasks)); }, [tasks]);

  const addTask = text => setTasks([...tasks, { id: Date.now(), text, done: false }]);
  const toggleTask = id => setTasks(tasks.map(t => (t.id === id ? { ...t, done: !t.done } : t)));
  const deleteTask = id => setTasks(tasks.filter(t => t.id !== id));
  const clearCompleted = () => setTasks(tasks.filter(t => !t.done));

  const visible = tasks.filter(t =>
   filter === 'All' ? true : filter === 'Active' ? !t.done : t.done);
  const remaining = tasks.filter(t => !t.done).length;

  return (
   <div className="app">
    <div className="card">
     <h1>📝 TODO App</h1>
     <AddTodo onAdd={addTask} />
     <TodoList tasks={visible} onToggle={toggleTask} onDelete={deleteTask} />
     <div className="footer">
      <span>{remaining} task{remaining !== 1 ? 's' : ''} left</span>
      <Filters filter={filter} setFilter={setFilter} />
      <button className="clear" onClick={clearCompleted}>Clear completed</button>
```

```
      </div>
    </div>
  </div>
 );
}

ReactDOM.createRoot(document.getElementById('root')).render(<TodoApp />);
</script>
</body>
</html>
```